

INSTITUTO FEDERAL
SÃO PAULO



Introdução ao Spring

Desenvolvimento Web Back-end (BRADWBK)

Prof. Luiz Gustavo Diniz de Oliveira Vêras

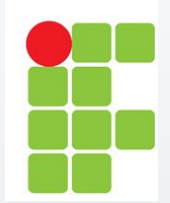
E-mail: gustavo_veras@ifsp.edu.br



Roteiro da aula

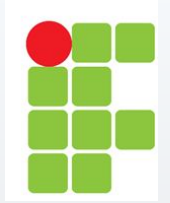
- ✓ **Spring Framework**
- ✓ **Criando um projeto do Spring no VS Code**
- ✓ **Conceitos básicos do Spring**
 - **Beans**

Spring Framework

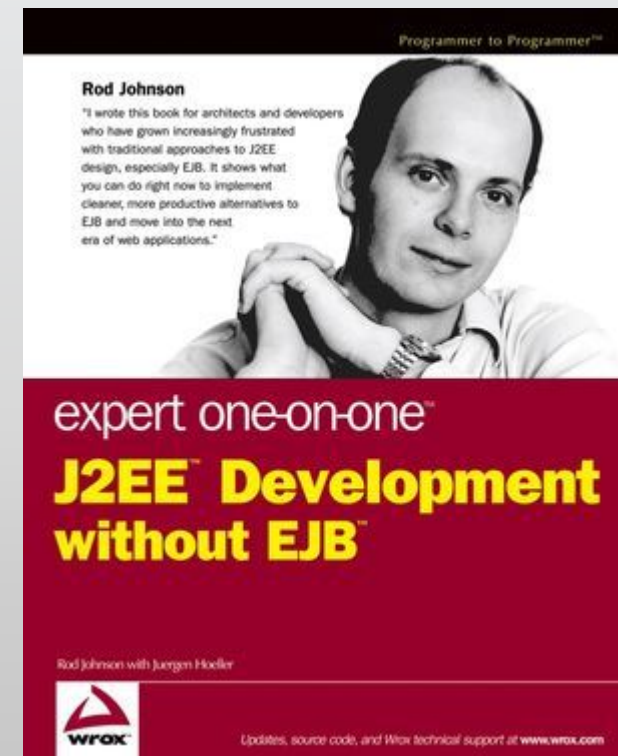


- O Spring Framework fornece um modelo abrangente de programação e configuração para aplicativos empresariais modernos (especialmente projetos para a Web) baseados em Java - em qualquer tipo de plataforma de implantação.
- **Um elemento-chave do Spring é o suporte de infraestrutura no nível do aplicativo:**
 - Assim as equipes de desenvolvimento podem se concentrar na lógica de negócios;
 - Evita esforços desnecessários com ambientes de implementação específicos na fase de desenvolvimento do aplicativo (programação);

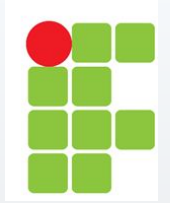
Spring Framework



- Início: Início dos anos 2000 com o livro de Rod Johnson.
- Ecossistema Java utilizava J2EE APIs, que eram difíceis de usar e testar.
- Spring veio para simplificar o ambiente Java.



Spring Framework



- Spring fornece suporte de infraestrutura para o desenvolvimento de aplicações Java corporativas.
- Spring lida com:



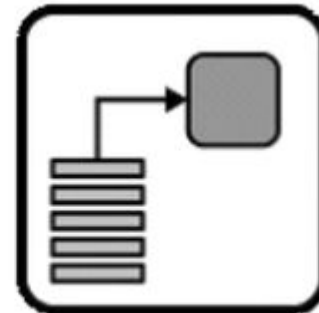
Web apps



Messaging



Persistence



Batch/Tasks

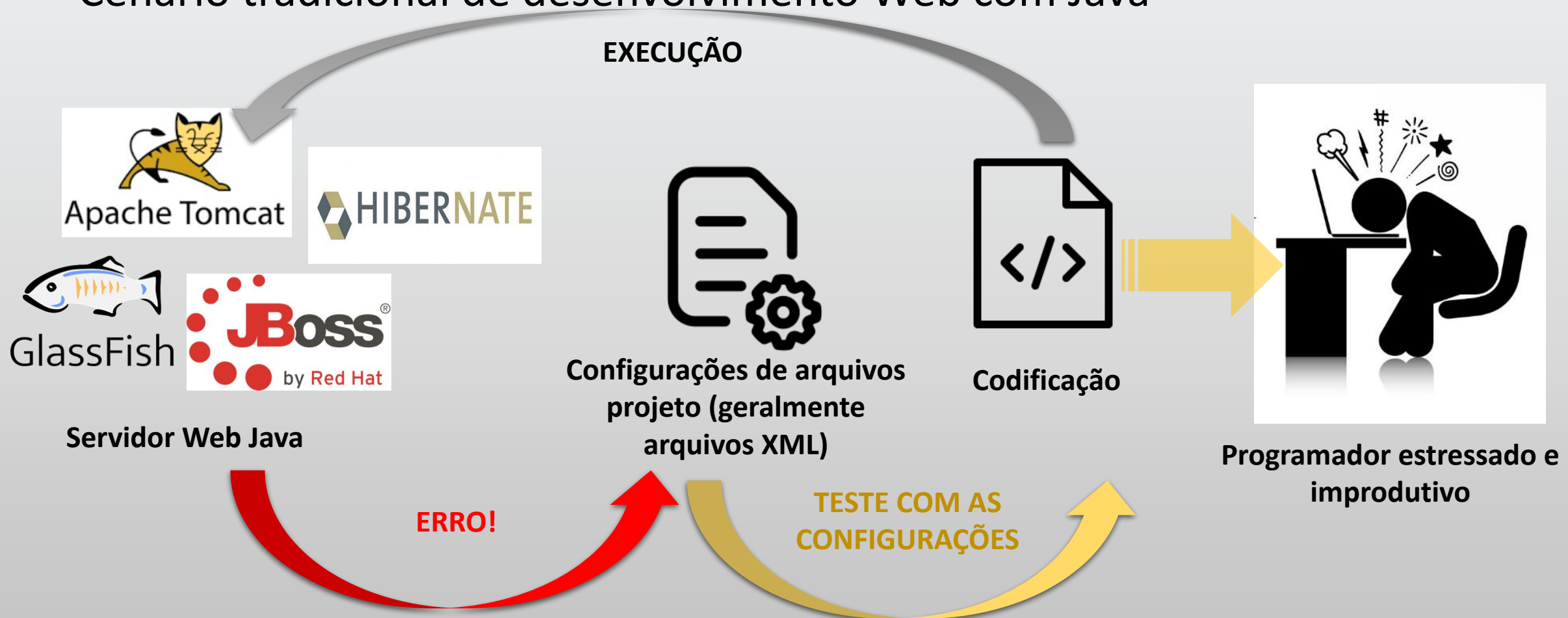


Integration/Streaming



Spring Framework

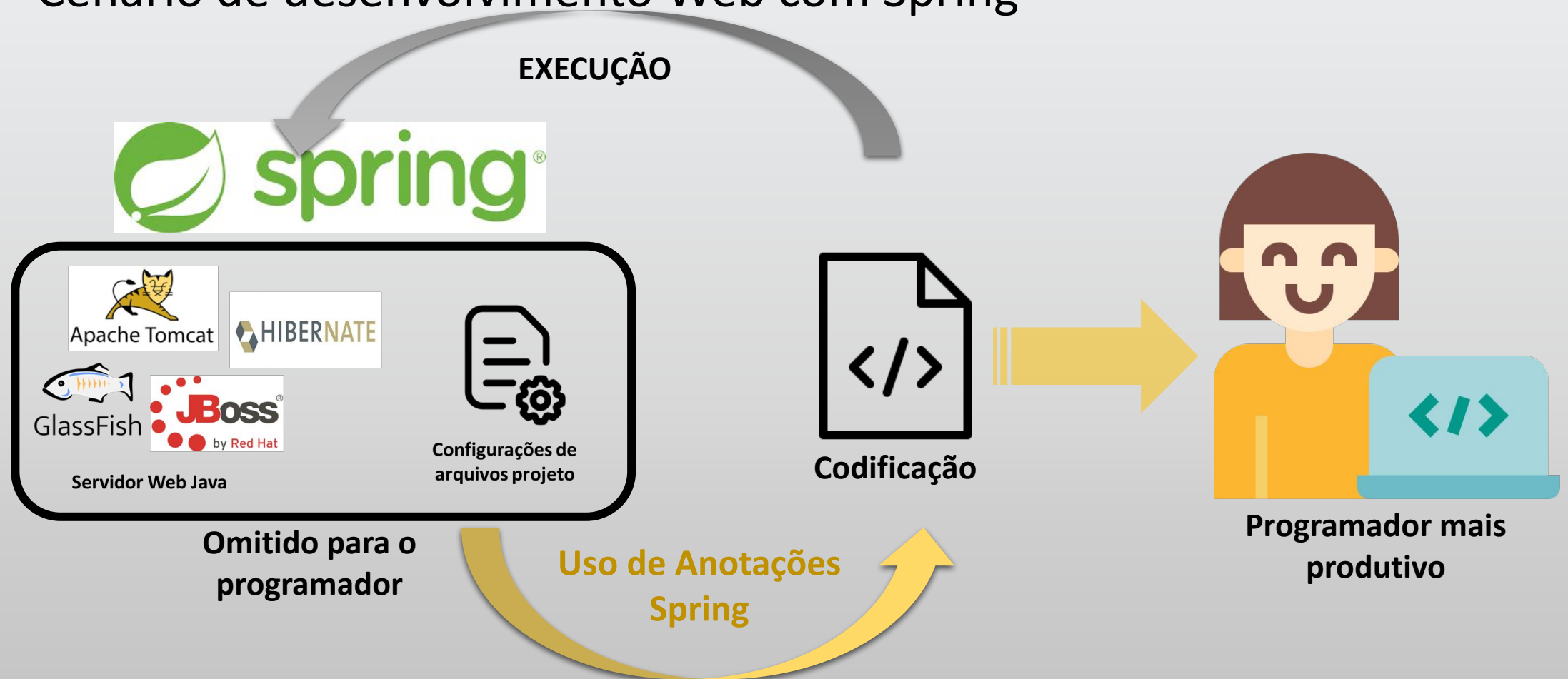
- Cenário tradicional de desenvolvimento Web com Java





Spring Framework

- Cenário de desenvolvimento Web com Spring





Família Spring

Alguns produtos da família Spring

- **Spring Boot:** Focado na experiência do desenvolvedor (DevEx). facilita a criação de aplicativos autônomos baseados em Spring de nível de produção que você pode "simplesmente executar" e utilizar recursos como *"hot reload"*.
- **Spring Web (Spring MVC):** fornece os recursos fundamentais para construção de aplicações web baseadas em arquitetura REST. Permite criar APIs RESTful utilizando anotações como *@RestController*, *@RequestMapping*, *@GetMapping*, entre outras, além de oferecer suporte a serialização automática (JSON/XML) e tratamento de requisições HTTP. Podemos aplicar **WebHooks** e **HTTPStreaming** com essa lib também.
- **Spring for GraphQL:** módulo oficial para integração de aplicações Spring com GraphQL. Permite expor APIs baseadas em consultas flexíveis, onde o cliente define exatamente quais dados deseja receber, reduzindo problemas de overfetching e underfetching comuns em REST.
- **Spring gRPC (via integração com gRPC Java):** embora não exista um módulo "core" chamado Spring gRPC, é possível integrar aplicações Spring Boot com gRPC utilizando bibliotecas como *grpc-spring-boot-starter*. Permite comunicação eficiente baseada em HTTP/2 e Protobuf, muito utilizada em arquiteturas de microsserviços.



Família Spring

Alguns produtos da família Spring

- **Spring WebSocket:** fornece suporte à comunicação bidirecional em tempo real entre cliente e servidor. Permite uso de WebSockets puros ou com protocolo STOMP sobre WebSocket, sendo muito utilizado em aplicações como chats, dashboards em tempo real e sistemas de notificação.
- **Spring Security:** Spring Security é uma estrutura de autenticação e controle de acesso poderosa e altamente personalizável. É o padrão de fato para proteger aplicativos baseados em Spring. Vamos utiliza-lo para configurar uma funcionalidade de Login.
- **Spring Data JPA:** visa melhorar significativamente a implementação de camadas de acesso a dados (Model), reduzindo o esforço para a quantidade que é realmente necessária de código para realizar persistência em banco de dados.



Família Spring

Alguns produtos

- **Spring Web**
entre cliente e servidor
STOMP e WebSocket
dashboards
- **Spring Security**
acesso por usuário e senha
aplicativo de autenticação
funcionalidade
- **Spring Data**
de acesso a dados
realmente

Veja a quantidade de produtos
oferecidos pela Spring

<https://spring.io/projects/spring-framework>

tempo real
protocolo
como chats,

controle de
proteger
criar uma

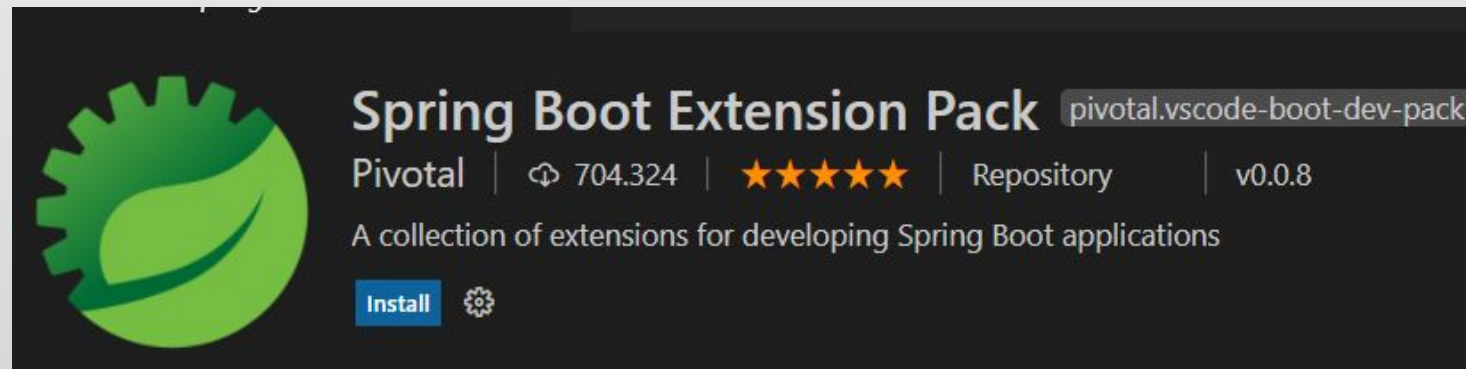
camadas
de que é
adados.



Trabalhando com Spring no VS Code

Primeiro, devemos instalar o plugin para trabalhar com o Spring

- Spring boot Extension pack



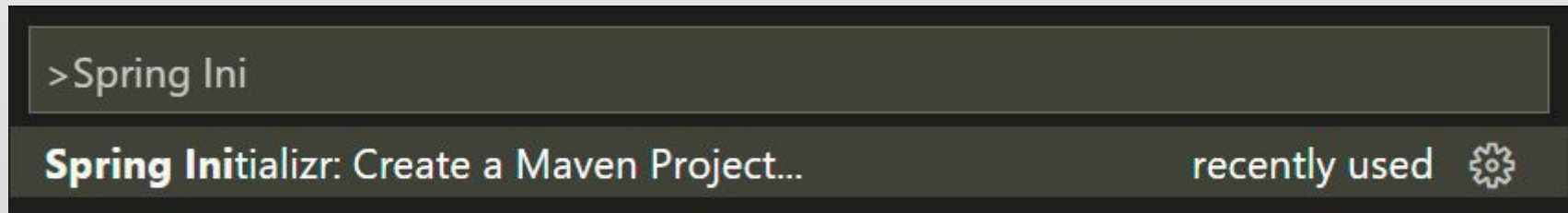


Trabalhando com Spring no VS Code

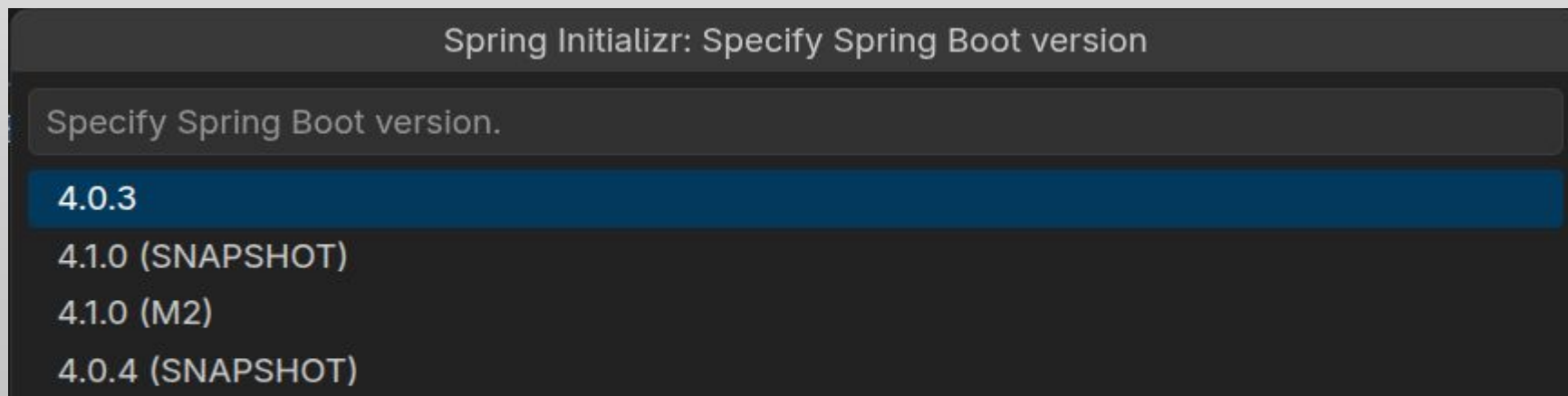
Criando o projeto

No VS Code, pressione CTRL + SHIFT + P.

- Digite *Spring Initializr* e Selecione a opção que contém *Maven Project*.



- Selecione a versão mais recente (que não tenha escrito SNAPSHOT)





Trabalhando com Spring no VS Code

Criando o projeto

- Selecione *Java* como linguagem para o projeto.

Specify project language.

Java

- O Spring solicita que você defina um id de grupo (algo que faça referência ao domínio da sua aplicação) para o projeto. Digite um nome com o padrão de nomeação de pacote.

com.ifsp

Input Group Id for your project. (Press 'Enter' to confirm or 'Escape' to cancel)

- O Spring solicita que você defina um id de artefato. Esse é o nome do projeto que irá trabalhar a partir daqui.

exemplo

Input Artifact Id for your project. (Press 'Enter' to confirm or 'Escape' to cancel)



Trabalhando com Spring no VS Code

Criando o projeto

- Selecione *Jar* o tipo de empacotamento.

Specify packaging type.

Jar

- Selecione a versão do Java que está instalado na sua máquina. Aqui usaremos a versão 11.

Specify Java version.

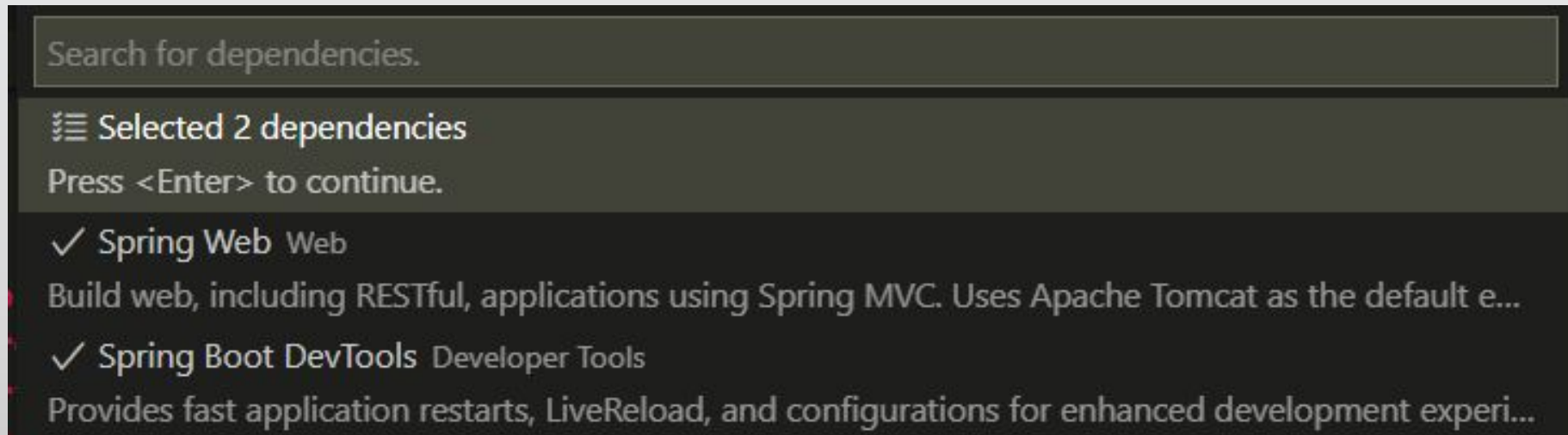
11



Trabalhando com Spring no VS Code

Criando o projeto

- Selecione as dependências (bibliotecas) que iremos utilizar.



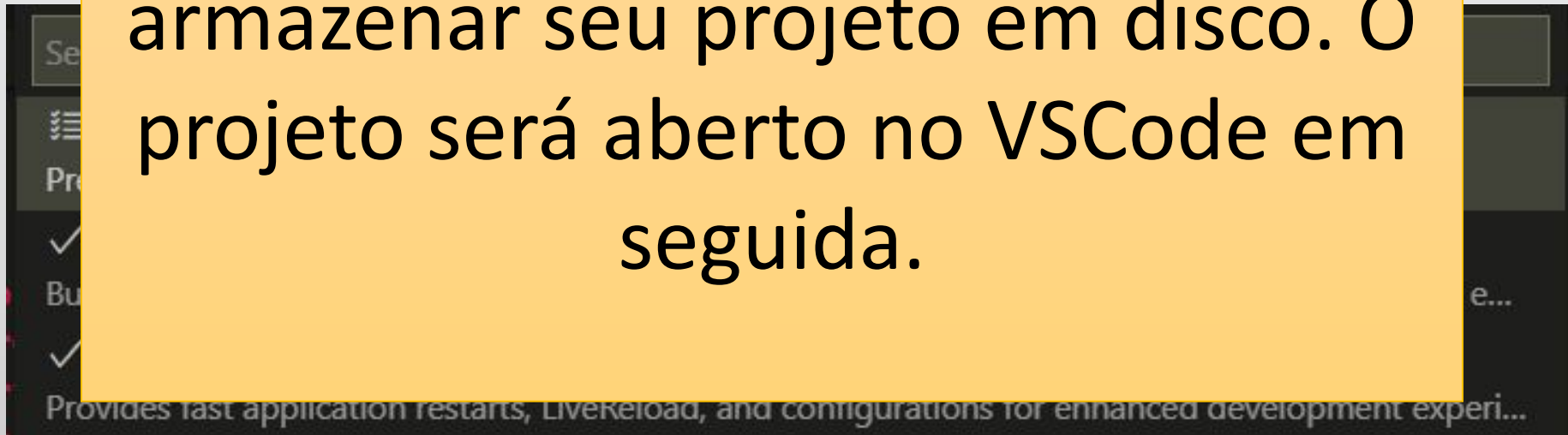


Trabalhando com Spring no VS Code

Criando o

- Selecion

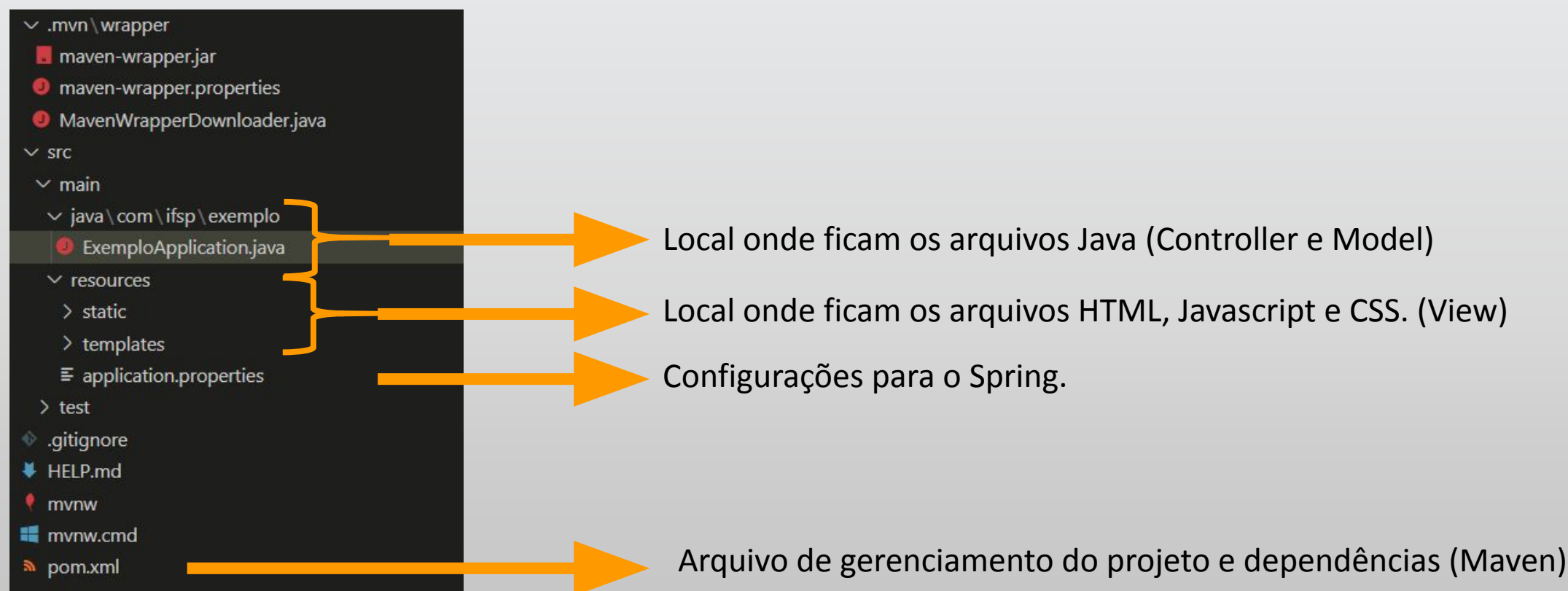
Por fim, selecione um local para armazenar seu projeto em disco. O projeto será aberto no VSCode em seguida.





Trabalhando com Spring no VS Code

Após selecionar o diretório onde o projeto será gerado, teremos a estrutura a seguir:





Trabalhando com Spring no VS Code

Abra o arquivo **ExemploApplication.java** e deixe como segue

```
@SpringBootApplication
@RestController
public class DemoApplication {

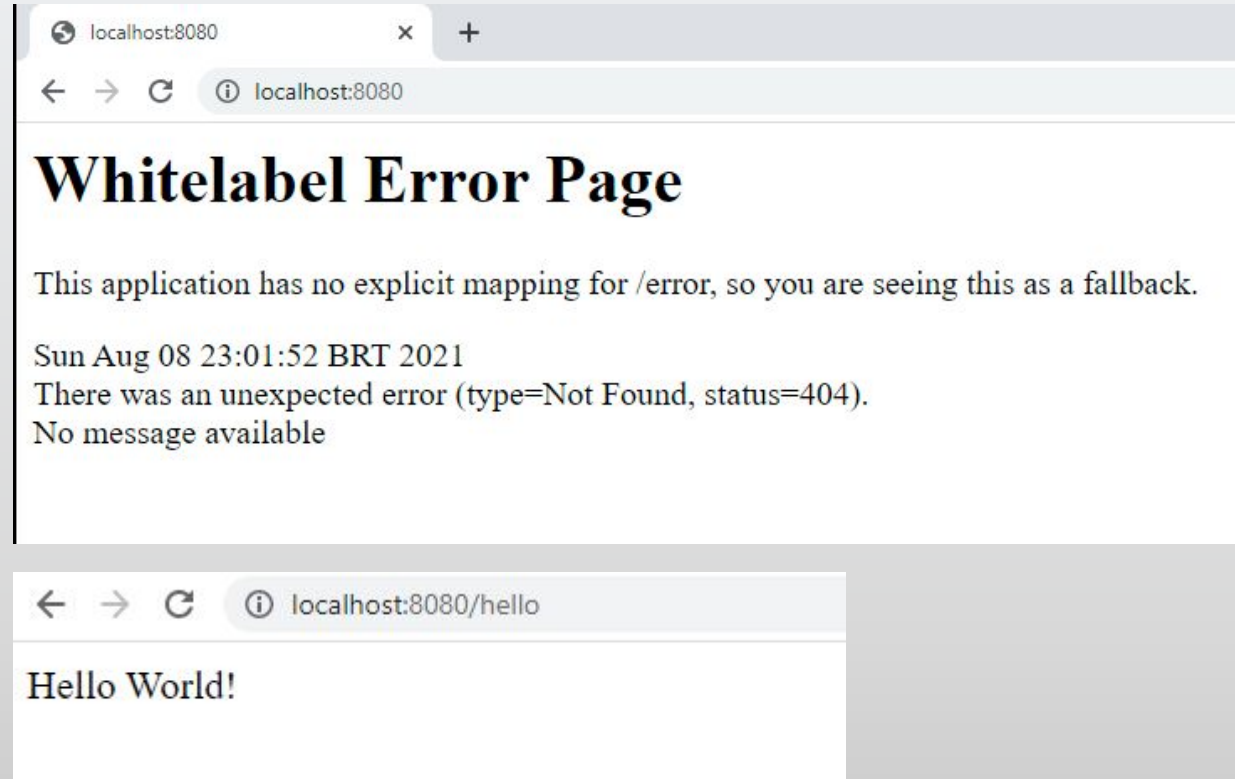
    @GetMapping("/hello")
    String home(){
        return "Hello World!";
    }

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```




Trabalhando com Spring no VS Code

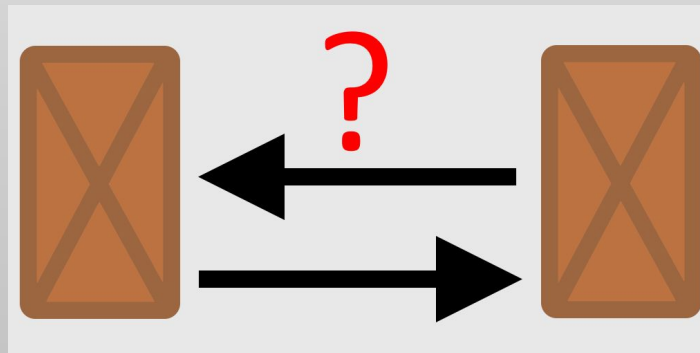
- Acesse no browser o endereço **localhost:8080** e veja o que aparece.
- Agora tente digitar **/hello** na frente do endereço e tente acessar.

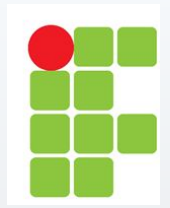




Conceitos de Spring Framework

- Spring é não invasivo:
 - O framework não vai obrigar você a estender classes ou implementar interfaces (na maioria dos casos)
 - Você escreve seu código por meio de POJOs.
- Baixa sobrecarga
 - Os projetos são empacotados no formato .jar, com tamanhos otimizados
- Spring instancia e injeta dependências de seus objetos
- Altamente extensível e customizável.
- Utiliza a Inversão de dependência para gerenciar os objetos da aplicação.





Conceitos de Spring Framework

O Spring está baseado em dois princípios de design:

- **Inversão de Controle (IoC - Inversion of Control)**

Neste princípio, a responsabilidade de controlar (criar, ativar, destruir) um objeto no sistema é transferida do programador para **um outro elemento (Spring Framework)**. A vantagem dessa abordagem é a facilidade em modificar e testar componentes do sistemas.

- **Injeção de Dependência (DI – Dependency Injection)**

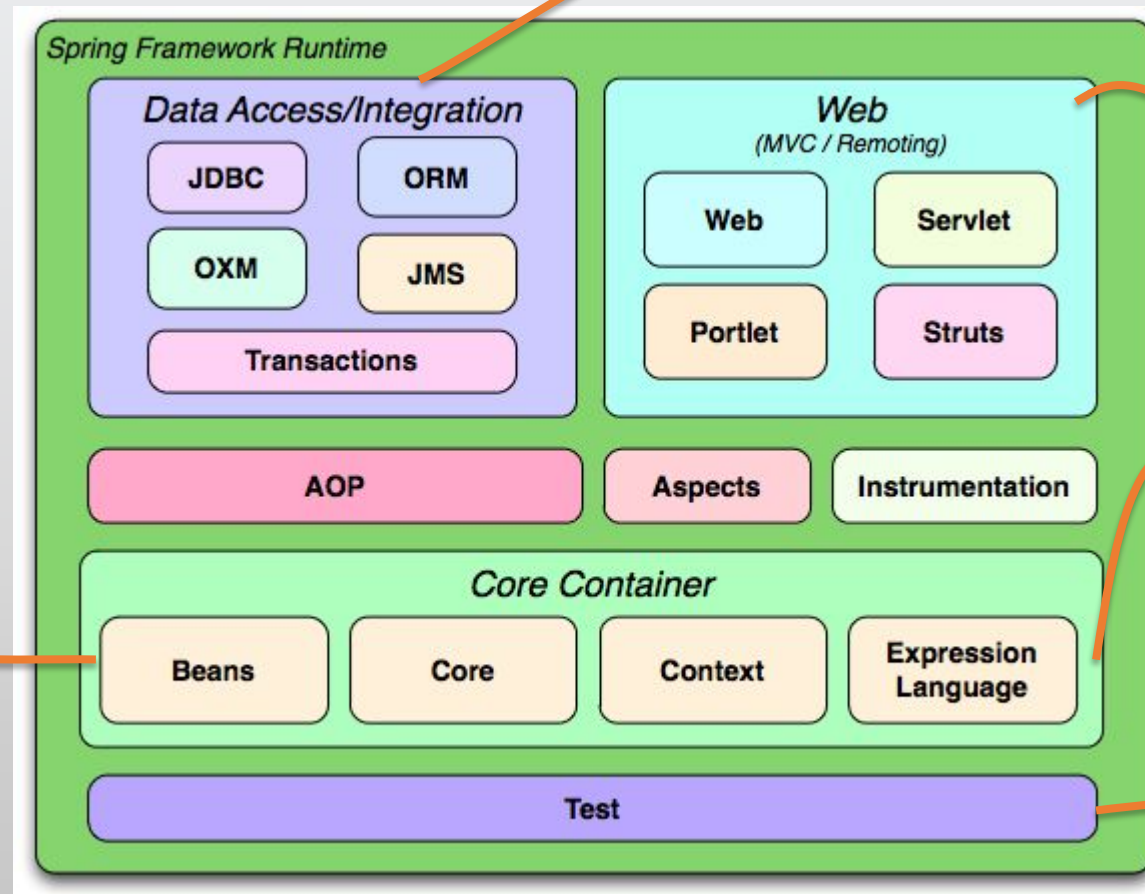
Neste princípio, um elemento do sistema é responsável por carregar (resolver dependências) para o programador, recuperando objetos na memória em uma parte específica de um sistema.

É a forma como o Spring implementa a inversão de controle. Dentre suas vantagens temos a facilidade em substituir dependências sem alterar o código principal, tornando o código mais organizado e testável.



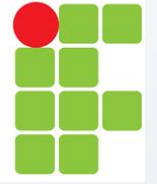
Conceitos de Spring Framework

Arquitetura do Spring Framework

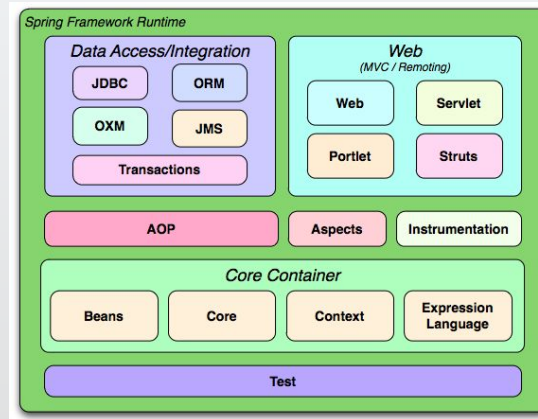


No Spring, um componente é criado em seu **container** (**Aplicação da Inversão de Controle**)...

... e carregado em diferentes partes da aplicação (**Aplicação da Injeção de Dependência**).



Conceitos de Spring Framework



- **Core Container:** Contém os módulos essenciais do Spring, incluindo:
 - **Beans:** Gerenciamento de beans (objetos gerenciados pelo Spring).
 - **Core:** Fornece funcionalidades básicas do framework (IoC e DI) .
 - **Context:** Suporte para injeção de dependências e acesso aos *beans*. Representa o container DI do Spring.
- **Data Access/Integration:** Facilita a comunicação com bancos de dados e sistemas de mensagens.
- **Web (MVC/Remoting):** Suporte para desenvolvimento web.
- **AOP (Aspect-Oriented Programming):** Permite programação orientada a aspectos (configuração de classes de logs, segurança e onde serão aplicadas pelo Spring).
- **Test:** Suporte para testes unitários e de integração no Spring.



Introdução aos Beans

A definição de **beans**, segundo a documentação do Spring é:

*“No spring, o objeto que forma a espinha dorsal da sua aplicação e que é gerenciada pelo container do Spring IoC é chamado de **beans**. Um **bean** é o objeto que é instaciado, montado e gerenciado pelo container do Spring IoC.”*

- Já existia no EJB (Enterprise JavaBeans) do J2EE
- Eram componentes de software reutilizáveis que estruturam a lógica de negócios e gerenciam o estado da aplicação



Introdução aos Beans

Definindo um **Spring Bean**

O método **myService()**, será utilizado pelo Spring para instanciar um objeto do tipo **MyService** que ficará no container do spring...

```
@Configuration
public class AppConfig {
    @Bean
    public MyService myService() {
        return new MyServiceImpl();
    }
}
```

... Esse objeto ficará disponível por toda a aplicação.



Introdução aos Beans

Tipos de Beans

No Spring existem alguns tipos de beans. Utilizando annotations como **@Component**, **@Configuration** ou **@Service** em suas classes e o **@Bean** em seus métodos, você passa a delegar o gerenciamento dessas dependências ao Spring.



Introdução aos Beans

@Component (O genérico)

É a anotação "mãe". Você a usa quando sua classe é um componente genérico que não se encaixa em nenhuma categoria específica (como uma classe utilitária).

```
@Component
public class GeradorDeProtocolo {
    public String criar() {
        return "PROT-" + System.currentTimeMillis();
    }
}
```



Introdução aos Beans

@Service (Regras de negócio)

É usada para classes que guardam a lógica principal do seu sistema. Na prática, faz o mesmo que o **@Component**, mas serve para organizar o código em serviços.

```
@Service
public class PedidoService {
    public void finalizarPedido() {
        System.out.println("Processando regras de pagamento e estoque...");
    }
}
```



Introdução aos Beans

@Configuration e @Bean (Criação manual dos objetos)

O **@Configuration** indica que a classe tem métodos que criam **Beans**. O **@Bean** é usado em cima de métodos, geralmente quando você precisa configurar um objeto de uma biblioteca externa (que você não pode editar para colocar um **@Component**).

```
@Configuration
public class AppConfig {

    @Bean
    public WebTemplate meuWebTemplate() {
        // Aqui você configura o objeto antes de entregar ao Spring
        return new WebTemplate();
    }
}
```



Introdução aos Beans

Em resumo

- **@Component, @Service, @Configuration:** Você coloca na sua classe para o Spring achá-la automaticamente (escaneamento).
- **@Bean:** Você usa dentro de uma classe de configuração para instanciar algo manualmente.

Injetamos os Beans em nosso código utilizando a anotação **@Autowired**.



Como utilizar o Spring

Exemplo

Em uma aplicação sem IoC o programador é responsável por instanciar cada objeto necessário para uma Tarefa...

```
public class Programa {  
  
    private Tarefa tarefa;  
  
    public void do() {  
        this.tarefa = new Tarefa();  
        tarefa.executar();  
    }  
}
```

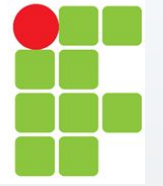



Como utilizar o Spring

Exemplo

...e também por determinar quando uma Tarefa deve ser executada.

```
public class Programa {  
    private Tarefa tarefa;  
  
    public void do() {  
        this.tarefa = new Tarefa();  
        tarefa.executar();  
    }  
}
```



Como utilizar o Spring

Exemplo

Na Inversão de Controle
podemos delegar a criação de
um objeto a outro elemento



No Spring chamamos esse elemento
de contêiner

E com isso uma classe pode se preocupar
menos em criar objetos e gerenciar suas
dependências e focar nas regras de negócio
que ela implementa

```
public class Programa {  
  
    private Tarefa tarefa;  
  
    public void do() {  
        tarefa.executar();  
    }  
}
```



Como utilizar o Spring

Exemplo

Como fazer isso no Spring?

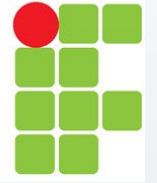
Na Inversão de Controle
podemos delegar a criação
um objeto a outro elemento

No Spring chamamos esse elemento
de contêiner

le se preocupar
gerenciar suas
regras de negócio
enta

```
grama {  
fa tarefa;
```

```
public void do() {  
    tarefa.executar();  
}  
}
```



Como utilizar o Spring

#1 – Defina uma interface
(boa prática em Java)

```
public interface Tarefa {  
    void executar();  
}
```

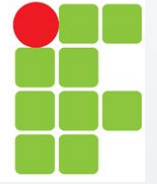
#2 – Crie uma classe que implemente a Tarefa e defina como componente com a anotação **@Component**.

```
import org.springframework.stereotype.Component;  
  
@Component  
public class TarefaImpl implements Tarefa {  
    @Override  
    public void executar() {  
        System.out.println("Executando tarefa...");  
    }  
}
```

#3 – Utilize-o em outro componente. O Spring injeta utilizando a anotação **@Autowired**. No caso abaixo, o objeto de **Tarefa** será aplicado ao parâmetro **tarefa**.

```
@Component  
public class Programa {  
    private final Tarefa tarefa;  
  
    @Autowired  
    public Programa(Tarefa tarefa) {  
        this.tarefa = tarefa;  
    }  
  
    public void executar() {  
        tarefa.executar();  
    }  
}
```

Obs: Cada código deve ser criado em um arquivo **.java** separado.



Como utilizar o Spring

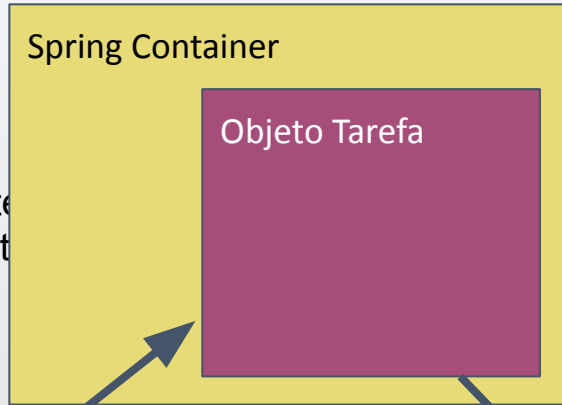
#1 – Defina uma interface
(boa prática em Java)

```
public interface Tarefa {  
    void executar();  
}
```

#2 – Crie uma classe que implemente
defina como componente com a anotação
@Component.

```
import org.springframework.stereotype.Component;  
  
@Component  
public class TarefaImpl implements Tarefa {  
    @Override  
    public void executar() {  
        System.out.println("Executando tarefa...");  
    }  
}
```

Spring
Instância



o em outro componente.
eta utilizando a anotação
@Autowired. No caso abaixo, o
tarefa será aplicado ao
tarefa.

Spring Injeta

```
@Component  
public class Programa {  
    private final Tarefa tarefa;  
  
    @Autowired  
    public Programa(Tarefa tarefa) {  
        this.tarefa = tarefa;  
    }  
  
    public void executar() {  
        tarefa.executar();  
    }  
}
```

Obs: Cada código deve ser
criado em um arquivo **.java**
separado.



Conceitos de Spring Framework

Altere o programa principal como segue.

```
@SpringBootApplication
public class Aplicacao implements CommandLineRunner {

    private final Programa programa;

    public Aplicacao(Programa programa) {
        this.programa = programa;
    }

    public static void main(String[] args) {
        SpringApplication.run(Aplicacao.class, args);
    }

    @Override
    public void run(String... args) {
        programa.executar();
    }
}
```

A saída “**Executando tarefa...**”
deve surgir no terminal.



Conceitos de Spring Framework

Exercício

Edite o exemplo anterior no Spring, utilizando outras anotações para configurar **beans** do Spring como segue:

- a) Altere a classe **TarefaImpl** e aplique a anotação **@Service (Slide 29)**. Executar novamente o projeto e verificar se a saída é a mesma.
- b) Agora remova a anotação de **TarefaImpl** e configure o bean para ser instanciado por uma classe de configuração **(Slide 30)**. Para tal, crie uma classe chamada **AppConfig** e aplique a anotação **@Configuration** junto **@Bean** em um método que retorna uma **TarefaImpl**. Execute o projeto novamente e verifique se a saída continua sendo apresentada.